

Bài thực hành số 1: Xây dựng và triển khai một hợp đồng thông minh cơ bản bằng Vyper

1. Mục tiêu

Bài thực hành nhằm giúp sinh viên nắm vững các kiến thức và kỹ năng cơ bản trong việc xây dựng và triển khai một hợp đồng thông minh trên Ethereum bằng ngôn ngữ Vyper. Sau khi hoàn thành, sinh viên có thể:

- Trình bày được nguyên lý hoạt động của blockchain và smart contract.
- Hiểu và giải thích các khái niệm cơ bản trong Vyper.
- Cài đặt và thiết lập môi trường phát triển Vyper.
- Viết và triển khai một hợp đồng thông minh cơ bản.
- Kiểm tra và xác minh chức năng của hợp đồng trên testnet.

2. Thời gian làm bài

Dự kiến: 3 giờ.

3. Cơ sở lý thuyết và bài tập tại lớp

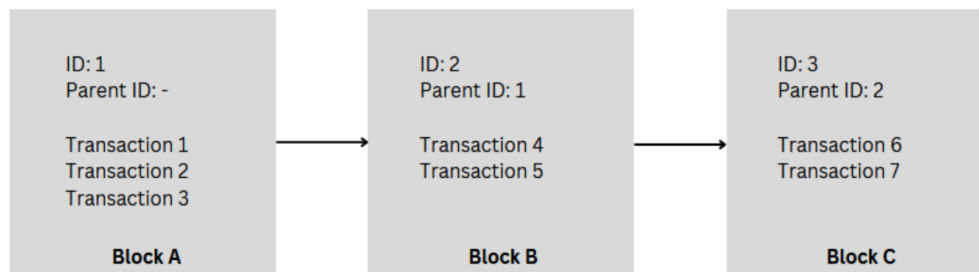
A. Blockchain

Blockchain là một cơ sở dữ liệu chỉ có thể thêm vào (append-only database) bao gồm các khối (blocks) được liên kết với nhau bằng cách sử dụng hàm băm (hashing). Trong đó, mỗi khối chứa nhiều giao dịch chuyển giá trị (nhưng cũng có thể là các loại thông tin khác) giữa những người tham gia, được bảo mật bằng mật mã học (cryptography). Một sự đồng thuận (consensus) giữa nhiều nút (nodes) nắm giữ cơ sở dữ liệu giống hệt nhau sẽ quyết định khối mới nào sẽ được thêm vào tiếp theo. Đặc tính nổi bật:

- Không thể thay đổi dữ liệu đã ghi.
- Minh bạch và có thể kiểm chứng.
- Được bảo mật nhờ mật mã học và cơ chế đồng thuận.

Để thêm một khối vào cơ sở dữ liệu phụ thuộc vào loại tiền mã hóa (cryptocurrency). Đối với Bitcoin, đó là lưu trữ các giao dịch. Ví dụ, Tý gửi một Bitcoin cho Tèo. Tuy nhiên, chúng ta sẽ tích lũy nhiều giao dịch vào một khối trước khi thêm vào cơ sở dữ liệu. Đối với Ethereum, những thứ có thể đưa vào cơ sở dữ liệu chỉ có thể thêm vào này càng nhiều. Nó không chỉ bao gồm việc chuyển giá trị – mà còn có thể là sự thay đổi trạng thái (change of state). Ví dụ, một hàng đợi để mua vé xem một buổi biểu diễn có thể là một trạng thái. Trạng thái này có thể là trống hoặc đầy. Tương tự như Bitcoin, trong Ethereum, tất cả các giao dịch có thể tập hợp trước khi thêm chúng vào cơ sở dữ liệu chỉ đọc. Ngoài danh sách các giao dịch, chúng ta còn lưu trữ những thứ khác trong

Bây giờ yếu tố 'khối' (block) của blockchain đã được giải thích, giờ hãy cùng xem xét yếu tố 'chuỗi' (chain). Như đã giải thích trước đó, ngoài danh sách các giao dịch, hàm băm của khối cha cũng được lưu vào khối. Nhưng tạm thời, một ID được sử dụng để chỉ tới khối cha thay vì sử dụng hàm băm. ID của khối cha chỉ là ID của khối trước đó. Ban đầu, không có khối nào. Thay vào đó, chúng ta đặt Block A, chứa ba giao dịch: Giao dịch 1, Giao dịch 2 và Giao dịch 3. Vì Block A là khối đầu tiên, nó không có khối cha. Sau đó, chúng ta thêm Block B vào sau Block A, bao gồm hai giao dịch: Giao dịch 4 và Giao dịch 5. Block B không phải là khối đầu tiên trong blockchain này. Do đó, chúng ta đặt phần khối cha trong Block B là ID của Block A vì Block A là khối cha của Block B. Sau đó, chúng ta đặt Block C vào blockchain, chứa hai giao dịch: Giao dịch 6 và Giao dịch 7. Phần khối cha trong Block C sẽ là ID của Block B, và cứ thế tiếp tục. Để đơn giản hóa, chúng ta tăng ID từ 0 lên 1 cho mỗi khối mới. Hình 1 dưới đây cho thấy ba khối được kết nối bằng trường ID của khối cha.



Thực hiện tạo một chuỗi khối đơn giản bằng ngôn ngữ python:

Thực hiện tạo một chuỗi khối đơn giản bằng ngôn ngữ python:

[illegible]

B. Chữ ký số lên dữ liệu trong blockchain

Trong blockchain, chúng ta sử dụng hai khóa để thực hiện chữ ký số lên dữ liệu nhằm mục đích xác thực một thông điệp và bảo vệ nó khỏi việc bị thay đổi bởi những người dùng trái phép: khóa riêng (private key) và khóa công khai (public key).

Khóa riêng được giữ bí mật và không được công khai. Mặt khác, khóa công khai được cung cấp cho phép mọi người đọc được dữ liệu.

Thực hiện tạo khóa riêng. Để làm điều này, chúng ta cần phần mềm OpenSSL. Bạn có thể cài đặt phần mềm này bằng cách chạy lệnh sau:

- Trên Linux thực hiện lệnh: `sudo apt-get install openssl`
 - Tạo khóa private key cho Tý: `openssl genrsa -out ty.pem 1024`
 - Tạo khóa public key cho Tý: `openssl rsa -in ty.pem -pubout > ty.pub`
- Trên Window thực hiện
 - Sử dụng Power Shell: cài đặt Chocolatey. Sau đó dùng lệnh để cài đặt OpenSSL: `choco install openssl`
 - Tải file thực thi .exe để cài đặt openssl, file nằm trên trang web: <https://github.com/openssl/openssl/wiki/Binaries>
 - Mở power shell gõ lệnh sau để tạo khóa công khai và khóa riêng tư: `openssl rsa -in ty.pem -pubout -out ty.pub`

Tạo môi trường ảo trên Python:

```
$ python3.10 -m venv .venv
$ source .venv/bin/activate
(.venv) $
```

Cài đặt thư viện cryptography cho ngôn ngữ Python: cryptography

Tạo một file python, đặt tên TySignature.py. Nạp thư viện và cấu hình:

```
from cryptography.hazmat.primitives import hashes
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.backends import default_backend
from cryptography.hazmat.primitives.asymmetric import rsa
from cryptography.hazmat.primitives import serialization

# Configuration
GENERATE_PRIVATE_KEY = False
DERIVE_PUBLIC_KEY_FROM_PRIVATE_KEY = False
PRIVATE_KEY_FILE = "ty.pem"
PUBLIC_KEY_FILE = "ty.pub"
MESSAGE = b"ty likes cat"
```

Thực hiện ký chữ số lên thông điệp:

```

if GENERATE_PRIVATE_KEY:
    # Generate private key
    private_key = rsa.generate_private_key(
        public_exponent=65537,
        key_size=2048,
        backend=default_backend()
    )
else:
    # Load private key from pem file
    with open(PRIVATE_KEY_FILE, "rb") as key_file:
        private_key = serialization.load_pem_private_key(
            key_file.read(),
            password=None,
            backend=default_backend()
        )
signature = private_key.sign(
    MESSAGE,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

```

Đọc dữ liệu từ thông điệp đã được bảo vệ bằng chữ ký số:

```

if DERIVE_PUBLIC_KEY_FROM_PRIVATE_KEY:
    # Getting public key from private key
    public_key = private_key.public_key()
else:
    # Load public key from file
    with open(PUBLIC_KEY_FILE, "rb") as key_file:
        public_key = serialization.load_pem_public_key(
            key_file.read(),
            backend=default_backend()
        )

public_key.verify(
    signature,
    MESSAGE,
    padding.PSS(
        mgf=padding.MGF1(hashes.SHA256()),
        salt_length=padding.PSS.MAX_LENGTH
    ),
    hashes.SHA256()
)

print(signature)

```

Thực hiện các yêu cầu sau:

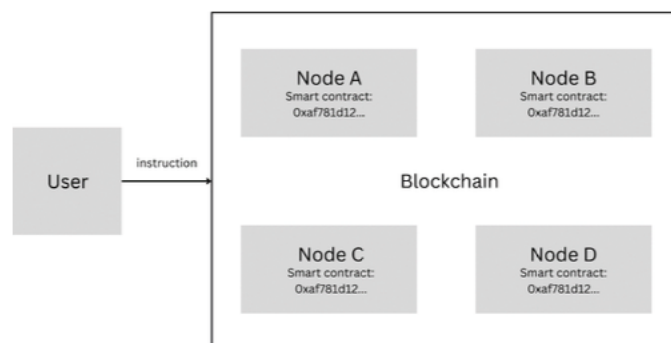
1. Bạn thử đóng vai trò của Tèo viết một đoạn chương trình Python để cố gắng sửa thông điệp của Tý thành “Tý hates cat”. Bạn chụp hình lại kết quả chạy.

2. Bạn chạy TySignature.py, chụp lại kết quả dữ liệu được mã hoá dựa trên thông điệp “Ty likes cat”. Sau đó bạn thay thông điệp thành “Ty hates cat” rồi chạy lại chương trình đồng thời chụp kết quả và so sánh 2 giá trị này.
3. * Bạn viết 1 chương trình để xác thực xem có phải Tý là người đã viết “Ty likes cat” hay không?

C. Smart contract

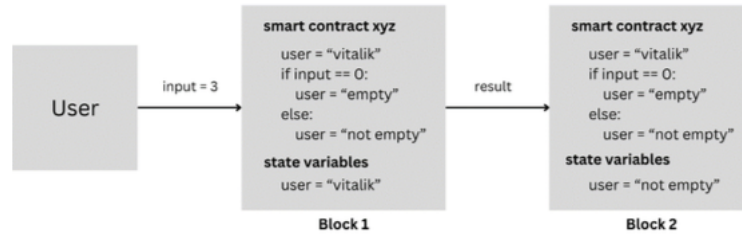
Smart contract (hợp đồng thông minh) là một loại chương trình khác được thiết kế để giải quyết các vấn đề chúng ta đã gặp phải trước đó. Trong khi ở Bitcoin chúng ta lưu trữ các giao dịch, chẳng hạn như Tý gửi 1 BTC cho Tèo, thì trong Ethereum, hợp đồng thông minh lưu trữ các giao dịch – ví dụ, ta có thể thay đổi giá trị của một biến từ 5 thành 9. Trong Bitcoin, các giao dịch rất hạn chế – chúng mang tính chất tài chính. Nhưng trong Ethereum, các giao dịch là những thay đổi trạng thái của các chương trình.

Hãy coi một hợp đồng thông minh như một chương trình hoặc một ứng dụng sống trên blockchain. Chương trình này dưới dạng bytecode dựa trên stack. Khi bạn tương tác với hợp đồng thông minh, bạn gửi một lệnh đến bytecode. Sau đó, máy ảo Ethereum sẽ đẩy và kéo các giá trị từ bytecode và thực hiện lệnh theo các giá trị từ bytecode. Vì vậy, toàn bộ chương trình của bạn sống trên blockchain. Bạn có thể hình dung một hợp đồng thông minh như một tệp nhị phân ứng dụng như tệp .exe trên hệ thống Windows, nhưng nó sống trên blockchain. Trong trường hợp này, bạn có thể xem xét vào bên trong chương trình. Điều này khác với một ứng dụng web, nơi tệp nhị phân chương trình bị ẩn khỏi bạn. Bạn có thể thấy rằng hợp đồng thông minh sống trong mỗi nút trên blockchain, như được hiển thị trong hình 2.



Hình 2. Một smart contract là một chương trình tồn tại trong chuỗi khối

Một người dùng có thể tương tác với hợp đồng thông minh và thay đổi trạng thái của chương trình, như hình số 3.

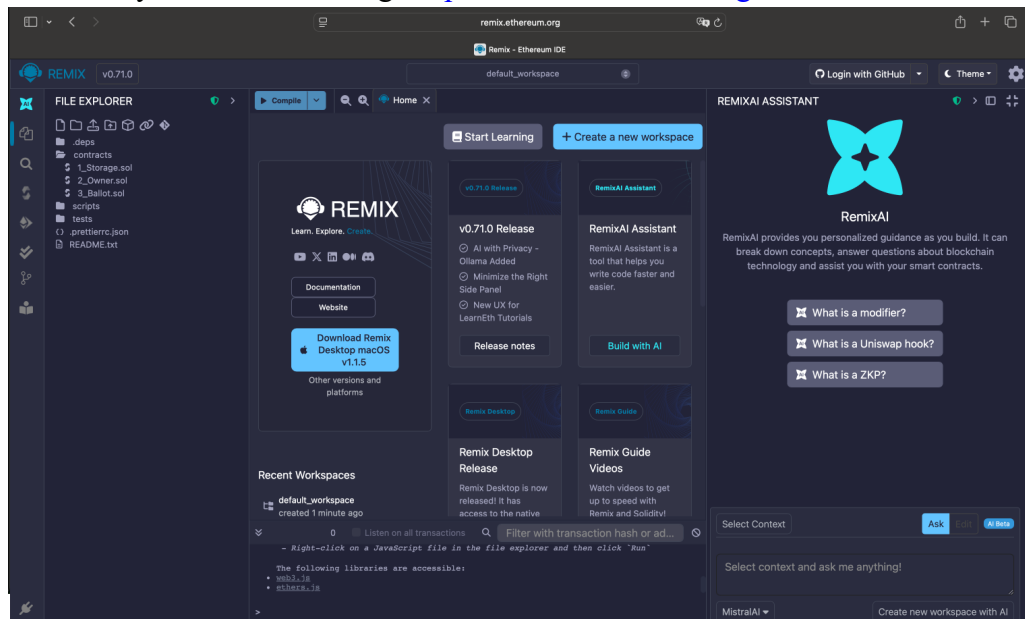


Hình 3. Người dùng gửi một input tới smart contract

D. Sử dụng Vyper hiện thực một hợp đồng thông minh

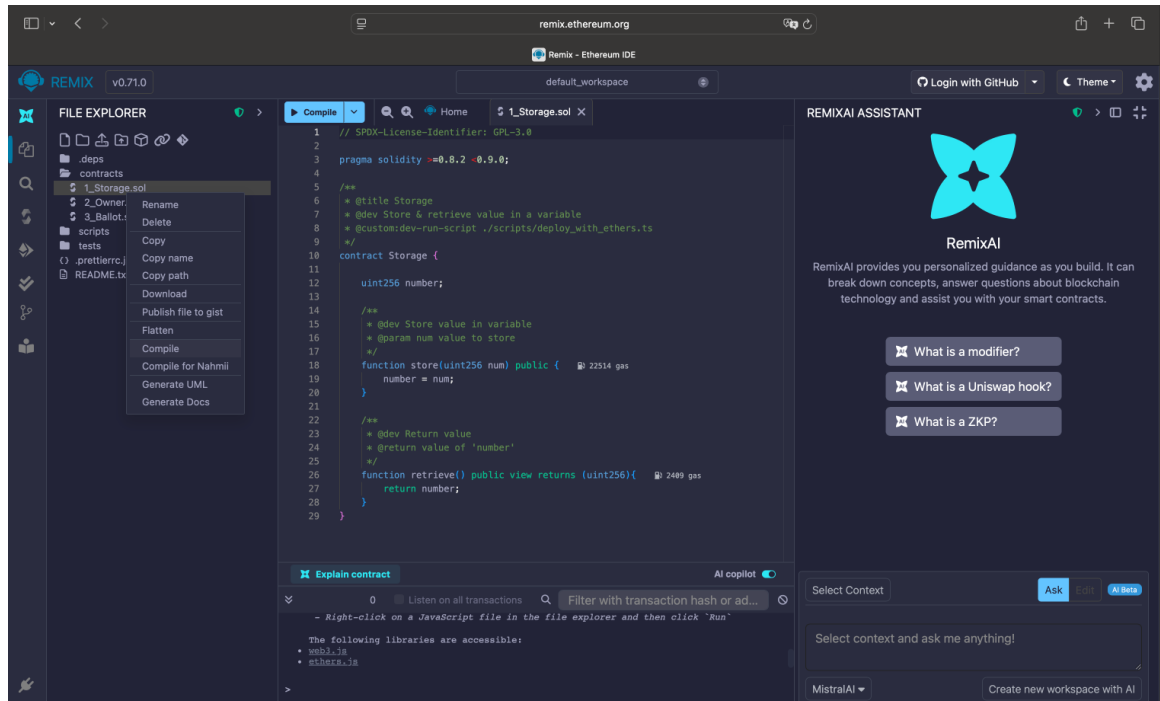
i. Môi trường triển khai smart contract trên Ethereum.

Mở trình duyệt web, đi tới trang: <https://remix.ethereum.org> như hình 4.



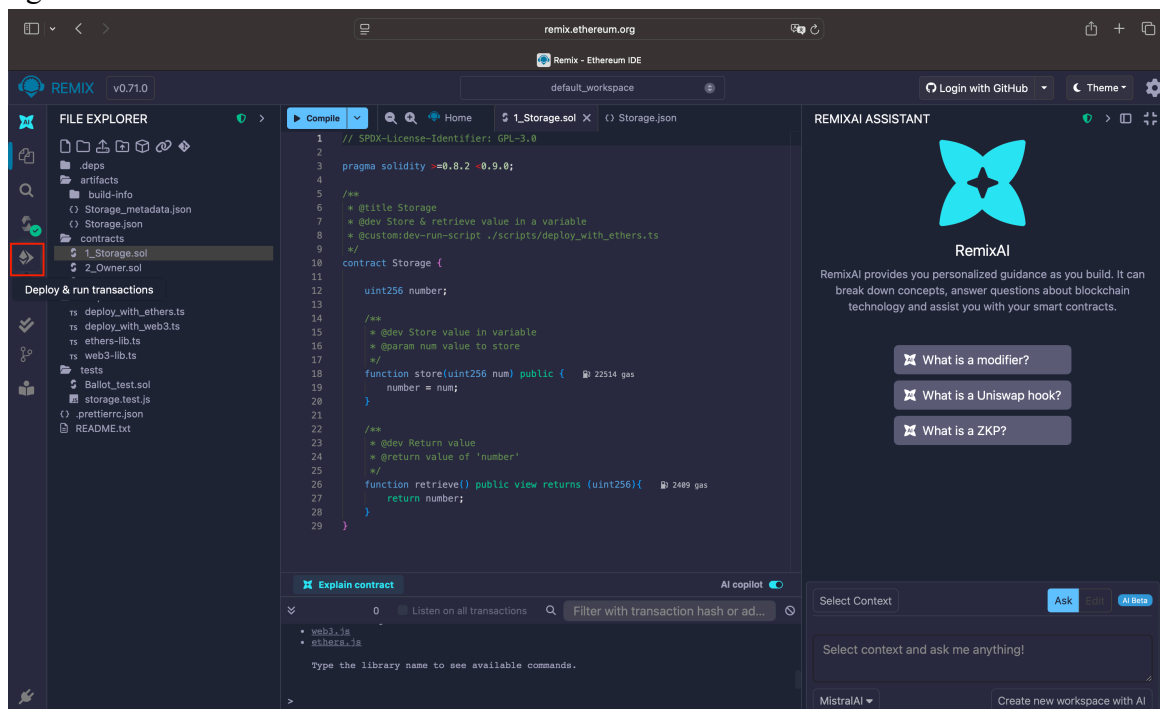
Hình 4. Remix

Cách biên dịch một chương trình hợp đồng thông minh. Click vào thư mục contracts trong cửa sổ FILE EXPLORER sau đó click chuột phải vào file 1_Storage.sol và chọn chức năng Compile như hình 5.



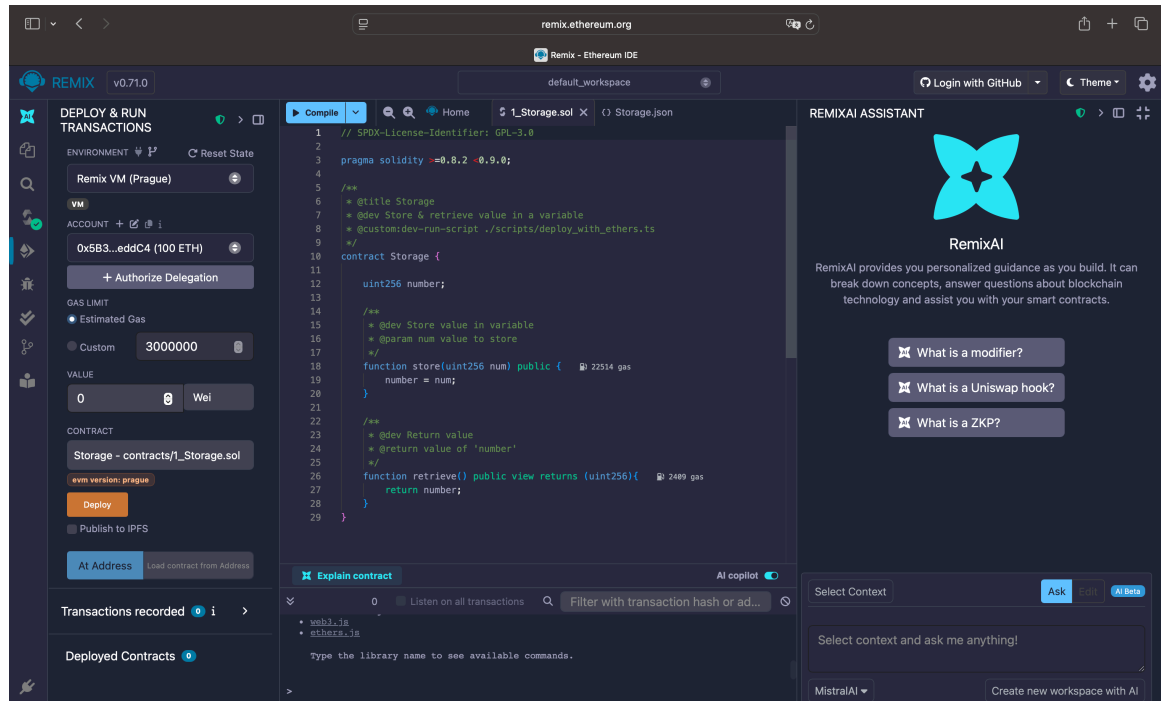
Hình 5. Biên dịch hợp đồng thông minh

Sau khi compile file hợp đồng thông minh, click vào file hợp đồng thông minh, click tiếp vào lệnh Deploy and run transactions trên menu để triển khai hợp đồng này lên nền tảng Ethereum như hình 6.



Hình 6. Triển khai một hợp đồng thông minh trên nền tảng Ethereum.

Kết quả khi triển khai hợp đồng thông minh 1_Storage.sol trên Ethereum như hình số 7.

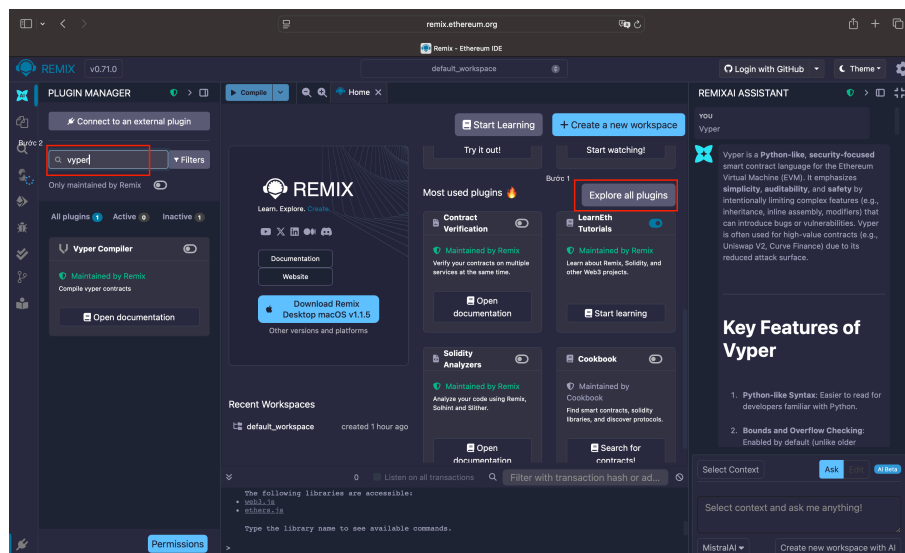


Hình 7. Kết quả triển khai hợp đồng thông minh

ii. Thiết lập Vyper

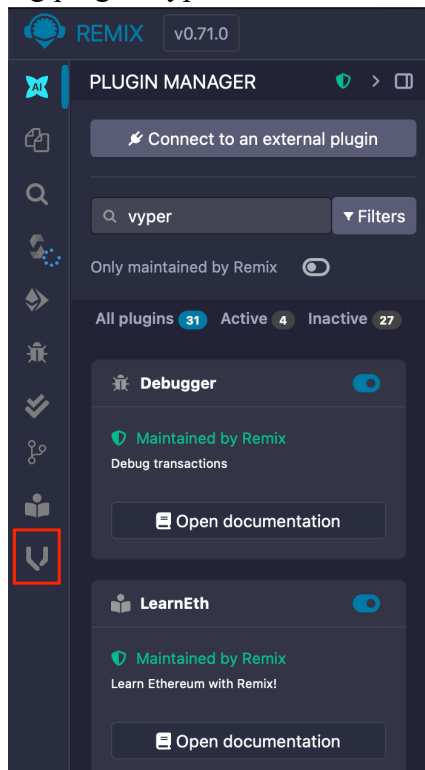
Chạy máy cá nhân. Sử dụng pip để cài đặt thư viện Vyper bằng câu lệnh: pip install vyper.

Chạy trên môi trường web. Mở trình duyệt web đi tới trang: . Thiết lập các bước như hình.



Hình 8. Thiết lập plugin Vyper chạy trên trang Remix.

Kết quả thiết lập thành công plugin Vyper như hình số 9.



Hình 9. Kết quả plugin compiler cho Vyper.

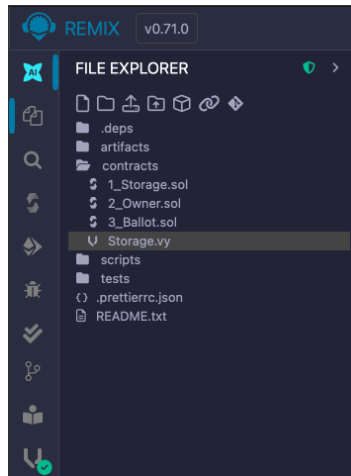
iii. Viết một hợp đồng đơn giản đầu tiên bằng Vyper

Các em tạo 1 file, đặt tên Storage.vy với nội dung như hình 10. Sau đó biên dịch rồi triển khai trên Remix như hình 11.

```
1  # @version ^0.4.3
2
3  number: uint256
4
5  @external
6  def store(num: uint256):
7      self.number = num
8
9  @external
10 def retrieve() -> uint256:
11     return self.number
```

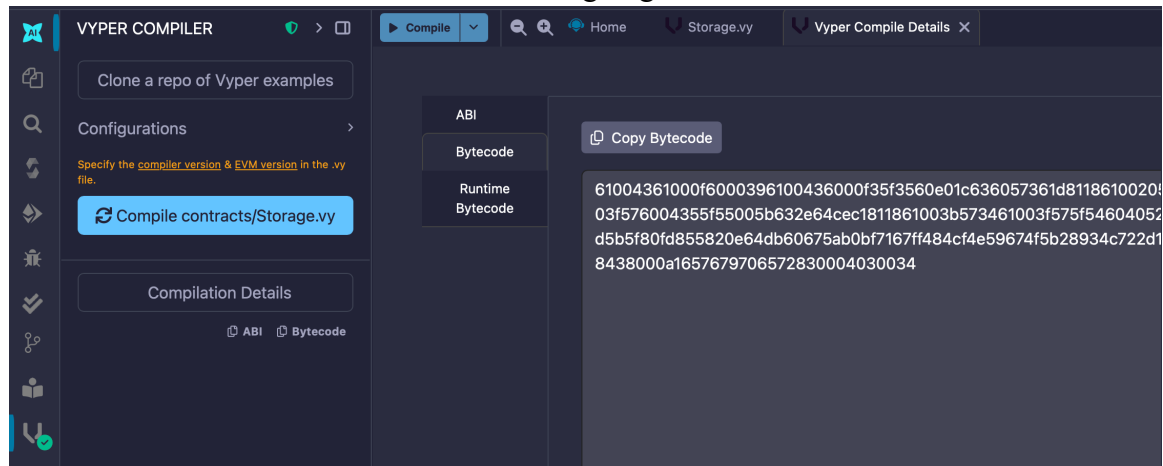
Hình 10. Hợp đồng thông minh đơn giản bằng Vyper.

Các em có thể biên dịch ở máy local bằng câu lệnh: `vyper -f abi,bytecode Storage.vy`



Hình 11. Upload file Storage lên Remix.

Sau khi biên dịch xong các em có thể xem mã bytecode của hợp đồng các em viết như hình 12, để triển khai các em có thể thực hiện giống như mục i.



Hình 12. Mã Bytecode của hợp đồng thông minh sau khi biên dịch

E. Ngôn ngữ lập trình Vyper

i. Kiểu dữ liệu

Các kiểu dữ liệu trong Vyper:

- Kiểu số nguyên có dấu
- Kiểu số nguyên không dấu
- Kiểu Boolean
- Địa chỉ
- Mảng kích thước cố định
- Mảng Byte
- Chuỗi
- Enum

- List
- Mảng động
- Struct
- Mapping

Viết một hợp đồng đặt tên `DataType.vy` để kiểm thử các kiểu dữ liệu như hình 13. Sau đó bạn biên dịch và chụp kết quả để nộp báo cáo.

```

1  # @version ^0.4.3
2
3  life_is_beautiful: bool
4  var_int1: int8
5  var_int2: int64
6  var_int3: int128
7  var_int4: int256
8  var_uint1: uint8
9  var_uint2: uint64
10 var_uint3: uint128
11 var_uint4: uint256
12 my_grandma_wallet: address
13 var_byte1: bytes32
14 var_byte2: bytes18
15 var_bytes: Bytes[56]
16 author: String[100]
17
18 flag Direction:
19     NORTH
20     SOUTH
21     WEST
22     EAST
23
24 direction: Direction
25
26 my_list: int16[5]
27 my_dynamic_array: DynArray[int128, 5]
28
29 struct Permission:
30     write: bool
31     execute: bool
32
33 my_permission: Permission
34 donatures: HashMap[address, uint256]

```

Hình 13. Kiểu dữ liệu trong ngôn ngữ Vyper

ii. Hàm (Functions)

Một hàm trong Vyper hoàn toàn giống như một hàm trong chương trình Python. Các câu lệnh được nhóm lại bên trong một khối mã và thực thi bất cứ lúc nào được gọi. Khối mã đó chính là hàm. Ví dụ như các hàm `store` và `retrieve` được viết trong file `Storage.vy`.

Giống như python khi chạy 1 file nếu có hàm `__init__` thì hàm này sẽ thực thi trước. Trong Vyper, một hàm cũng cung cấp chức năng tương tự. Tạo một file `StorageInit.vy` như hình 14.

```

1  # @version ^0.4.3
2
3  my_grandma_wallet: address
4  author: String[100]
5  fib_list: int16[5]
6  fib_dynamic_array: DynArray[int128, 5]
7  struct Permission:
8      write: bool
9      execute: bool
10 my_permission: Permission
11 donaturs: HashMap[address, uint256]
12
13
14 @deploy
15 def __init__():
16     self.my_grandma_wallet = 0xde93510CFa39Ab92BF927399F799DbE71997Ee0b
17     self.author = "Arjuna Sky Kok"
18     self.fib_list = [1, 1, 2, 3, 5]
19     self.fib_dynamic_array.append(1)
20     self.fib_dynamic_array.append(1)
21     self.fib_dynamic_array.append(2)
22     self.my_permission = Permission({write: True, execute: False})
23     self.donaturs[self.my_grandma_wallet] = 4000000000000000000
24
25     donation_target : int128 = 8000000000000000000

```

Hình 14. Hàm khởi tạo trong Vyper

iii. Mutability

Các chú thích (annotations) được sử dụng ở các ví dụ trước đó như, `@external` và `@internal`, là dùng cho mục đích phạm vi của hàm. Nhưng còn có nhiều chú thích với mục đích khác, chẳng hạn như khả năng biến đổi (mutability). Tính biến đổi ở đây nghĩa là hàm đó có thay đổi trạng thái của hợp đồng thông minh và nhận thanh toán hay không.

Nếu một hàm không thay đổi trạng thái của hợp đồng thông minh, chú thích `@pure` hoặc chú thích `@view` có thể được sử dụng. Chú thích `@pure` cho biết hàm đó không đọc các biến trạng thái và các biến môi trường. Chú thích `@view` cho biết hàm đó có đọc các biến trạng thái và biến môi trường nhưng không thay đổi các biến trạng thái.

Nếu ta muốn viết một hàm làm thay đổi các biến trạng thái, ta có thể sử dụng chú thích `@nonpayable` và chú thích `@payable`. Điểm khác biệt giữa hai chú thích này là các hàm có chú thích `@payable` có thể nhận hoặc trả thanh toán, trong khi các hàm có chú thích `@nonpayable` thì không thể.

Trong một chương trình, ta có thể nhận và lưu trữ thông tin thanh toán bên trong nó. Đây là một trong những tính chất độc đáo của hợp đồng thông minh. Một hợp đồng thông minh có thể hoạt động như một dịch vụ ký quỹ (escrow) hoặc một ngân hàng.

Bây giờ, ta sẽ tạo một chương trình đơn giản để minh họa cho các chú thích này. Tạo một file có tên là `Annotations.vy` và thêm đoạn mã sau vào như hình 15.

```

# @version ^0.4.3

author: String[100]
donatur: String[100]

@deploy
# đã ngam định @external
def __init__():
    self.author = "Arjuna Sky Kok"

@external
@pure
def add(x: int128, y: int128) -> int128:
    return x + y

@external
@view
def get_name_and_title() -> String[200]:
    return concat("Mr. ", self.author)

@external
@nonpayable
def change_name(new_name: String[100]):
    self.author = new_name

@external
@payable
def donate(donatur_name: String[100]):
    self.donatur = donatur_name

```

Hình 15. Ví dụ sử dụng các chú thích và công dụng của nó cho hàm

Yêu cầu: Biên dịch, sau đó triển khai đoạn hợp đồng thông minh trên nền tảng Ethereum để xem kết quả như thế nào, và ghi nhận vào báo cáo?

iv. Cấu trúc điều khiển

Giống như ngôn ngữ lập trình Python, Vyper cũng có các câu lệnh if, và for như hình 16.

```

# @version ^0.4.3

@external
@pure
def sum() -> uint256:
    s: uint256 = 0
    for i: uint256 in [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]:
        s += i
    return s

@external
@pure
def greet(time: String[10]) -> String[20]:
    if time == "morning":
        return "Good morning!"
    elif time == "evening":
        return "Good evening!"
    else:
        return "How are you?"

```

Hình 16. Ví dụ về cấu trúc câu lệnh điều khiển.

Yêu cầu: Biên dịch, sau đó triển khai đoạn hợp đồng thông minh trên nền tảng Ethereum để xem kết quả như thế nào, và ghi nhận vào báo cáo?

v. Biến môi trường

Có các biến môi trường mô tả các thuộc tính của khối (block) và giao dịch (transaction). Nếu ta muốn xác định đang ở khối nào ngay lúc này (giả sử trong blockchain đang có nhiều hơn một hợp đồng thông minh tồn tại), ta có thể lấy thông tin đó từ `block.number`. Nếu muốn biết số lượng ethereum mà một người dùng gửi đến hợp đồng thông minh, ta có thể sử dụng `msg.value`. Nếu ta muốn biết ai là người gửi tiền, ta có thể truy vấn thông tin đó từ `msg.sender`. Nếu ta muốn lưu trữ thời điểm một người dùng gửi ethereum, ta có thể sử dụng `block.timestamp`.

Để minh họa cho các biến môi trường, ta tạo một file Vyper, đặt tên là `EnvVar.vy`; có nội dung như hình 17.

```
1  # @version ^0.4.3
2
3  donatur: address
4  donation: uint256
5  time: uint256
6
7  @external
8  @payable
9  def donate():
10     self.donatur = msg.sender
11     self.donation = msg.value
12     self.time = block.timestamp
```

Hình 17. Đoạn chương trình ví dụ biến môi trường

Yêu cầu: Biên dịch, sau đó triển khai đoạn hợp đồng thông minh trên nền tảng Ethereum để xem kết quả như thế nào, và ghi nhận vào báo cáo?

vi. Event logging

Bên cạnh việc lưu trữ thông tin trong các biến trạng thái, ta cũng có thể ghi nhật ký thông tin bằng cách lưu lại các sự kiện (event logging).

Điều này rất hữu ích cho những người muốn theo dõi hoặc truy vết một số sự kiện. Ta có thể theo dõi các sự kiện cụ thể trên blockchain và nếu có một sự kiện mà ta quan tâm, ta có thể muốn thực hiện một điều gì đó, chẳng hạn như thông báo trên một trang web mạng xã hội rằng mục tiêu quyên góp đã đạt được.

Để hiểu về cách ghi nhật ký sự kiện, ta hãy viết một hợp đồng thông minh đơn giản. Đầu tiên, tạo một file Vyper, file này có tên là `EventLogging.vy`, sau đó thực hiện đoạn chương trình như hình 18.

```

1  # @version ^0.4.3
2
3  event Donation:
4      donatur: indexed(address)
5      amount: uint256
6
7
8  @external
9  @payable
10 def donate():
11     log Donation(msg.sender, msg.value)

```

Hình 18. Đoạn chương trình thực hiện EventLog

Yêu cầu: Biên dịch, sau đó triển khai đoạn hợp đồng thông minh trên nền tảng Ethereum để xem kết quả như thế nào, và ghi nhận vào báo cáo?

vii. Interface

Ta đã tương tác với một hợp đồng thông minh với tư cách là người dùng (user), nhưng một hợp đồng thông minh cũng có thể tương tác với một hợp đồng thông minh khác bằng cách sử dụng một interface.

Ta tạo một hợp đồng thông minh tương tác với hợp đồng thông minh Storage.vy đã được triển khai trước đó ở mục iii, bằng cách tạo tập tin StorageClient.vy và thêm đoạn mã chương trình như hình 19.

```

1  # @version ^0.4.3
2
3  interface Storage:
4      def retrieve() -> uint256: view
5
6  storage_contract: Storage
7
8  @deploy
9  def __init__(storage_address: address):
10     self.storage_contract = Storage(storage_address)
11
12  @external
13  def call_retrieve() -> uint256:
14     return staticcall self.storage_contract.retrieve()

```

Hình 19. Đoạn chương trình minh họa interface

Yêu cầu: Biên dịch, sau đó triển khai đoạn hợp đồng thông minh trên nền tảng Ethereum để xem kết quả như thế nào, và ghi nhận vào báo cáo?

4. Bài tập về nhà

Bài 1: Kiểu dữ liệu & hàm cơ bản

Viết chương trình hợp đồng thông minh lưu trữ số nguyên. Yêu cầu hợp đồng cho phép

- Lưu một số nguyên (int128) vào biến trạng thái.
- Lấy lại giá trị đã lưu.

Hướng dẫn làm bài: viết 2 hàm: một hàm là store dùng để lưu giá trị, một hàm là retrieve dùng để trả về giá trị đã lưu

Bài 2: Thanh toán ETH & quản lý số dư

Viết chương trình hợp đồng thông minh quản lý người gửi tiền. Yêu cầu hợp đồng cho phép:

- Người dùng gửi ETH vào hợp đồng (hàm deposit).
- Lưu trữ số dư của từng địa chỉ.
- Cho phép người dùng rút tiền của mình (hàm withdraw).

Hướng dẫn làm bài: viết 3 hàm: một hàm là deposit() thực hiện chức năng gửi, một hàm là balances(<địa chỉ>) kiểm tra số dư, và hàm thứ ba là withdraw() thực hiện chức năng rút tiền.

Bài 3: Cấu trúc điều khiển & HashMap

Viết chương trình hợp đồng thông minh bỏ phiếu đơn giản. Yêu cầu hợp đồng cho phép:

- Có một danh sách ứng cử viên.
- Mỗi địa chỉ chỉ được bỏ phiếu một lần.
- Xác định người thắng cuộc.

Hướng dẫn làm bài: Sử dụng mảng tạo danh sách ứng viên: có thể gán 3 ứng viên mặc định là Tý, Tèo, Mai. Viết hàm vote(name_candidate) để user thực hiện bỏ phiếu; và viết hàm getWinner() để trả về người có số phiếu cao nhất.

Bài 4: Struct & tra cứu dữ liệu

Viết chương trình hợp đồng thông minh quản lý danh bạ. Yêu cầu hợp đồng cho phép:

- Lưu danh sách liên hệ với thông tin: tên, số điện thoại.
- Cho phép thêm liên hệ mới.
- Tra cứu số điện thoại theo tên.

Hướng dẫn làm bài: Sử dụng cấu trúc struct để lưu thông tin liên hệ có 2 trường là tên và số điện thoại. Viết 2 hàm: một hàm là addContract(name, phone) để lưu một liên hệ mới, hàm thứ hai là getPhone(name) trả về số điện thoại liên hệ người tên name.

Bài 5: Xử lý logic và biến môi trường.

Viết chương trình hợp đồng thông minh gây quỹ cộng đồng (Crowdfunding). Yêu cầu hợp đồng cho phép:

- Chủ dự án tạo hợp đồng với mục tiêu và deadline.
- Người dùng có thể quyên góp bằng ETH.
- Nếu đạt mục tiêu trước deadline → chủ dự án rút tiền.
- Nếu không đạt → người dùng có thể rút lại tiền.

Hướng dẫn làm bài: Có thể thiết lập mục tiêu là 5 ETH và thời gian thực hiện là 10'. Viết hàm contribute cho phép người gửi tiền. Viết hàm withdraw thực hiện kiểm tra nếu số dư đạt mục tiêu trước và đúng thời hạn, người gây quỹ rút tiền, ngược lại nếu không đạt được sau 10', những người đóng góp được rút tiền.

Bìa 6: (Bài tập nâng cao) Hợp đồng Quản lý Tài liệu & Xác thực Chữ ký Số (Document Notary).

Tình huống: Trong cuộc sống, khi một cá nhân hoặc tổ chức muốn xác thực tính toàn vẹn của một tài liệu (ví dụ: hợp đồng lao động, giấy chứng nhận quyền sở hữu trí tuệ), họ có thể sử dụng Blockchain để công chứng (notarization) bằng cách:

- Tính hash của tài liệu.
- Lưu hash vào smart contract.
- Gắn kèm chữ ký số của chủ sở hữu tài liệu.
- Người khác có thể xác minh tài liệu đó có đúng bản gốc không bằng cách so sánh hash.

Yêu cầu bài tập: Sử dụng ngôn ngữ Vyper viết một smart contract để quản lý công chứng tài liệu với các chức năng sau:

- Đăng ký tài liệu
 - Người dùng gửi hash của tài liệu (bytes32).
 - Lưu thông tin: hash, người đăng ký, timestamp.
 - Sinh event DocumentRegistered.
- Xác thực tài liệu
 - Cho phép người khác kiểm tra xem tài liệu với hash cụ thể có tồn tại không.
- Quản lý chữ ký số
 - Người dùng có thể đính kèm một chữ ký (string, tối đa 256 ký tự) cho tài liệu của họ.
 - Sinh event SignatureAdded.
- Bảo mật
 - Chỉ người đăng ký tài liệu mới có quyền thêm chữ ký số cho tài liệu đó.
- Yêu cầu nâng cao
 - Sử dụng block.timestamp để lưu thời điểm đăng ký.
 - Sử dụng event logging để ghi lại toàn bộ lịch sử đăng ký và ký số.